Manual Repetitive Work and the Perfect AI Employee

Manual repetitive work across the world - and the perfect AI Employee for it

Owner's ask (5 Nov 2026): *"NOW DO DEEP RESEARCH ON MANUAL REPETITIVE WORK IN WORLD AND DEFINE A PERFECT AI EMPLOYEE FOR THAT"*. Twenty-fifth in the series, and a different kind: the 24 industry registers each listed jobs; this one asks what is **the same job in all of them**. Reading the registers side by side, a few dozen "AI Employees" turn out to be the same dozen *kinds of repetitive work* wearing different industry names (a clinic's no-show chase, a gym's renewal reminder and a landlord's rent follow-up are one job). This file defines that dozen, what the evidence says about automating it, why past automation kept breaking, and then **specifies one perfect AI Employee** that does repetitive work well in any industry.

How honest this is (read first)

- Five web searches were run this session. They returned a mix of reports and vendor blogs. **What a source actually showed is linked; everything else is my synthesis** from the 24 earlier registers and general knowledge, and is labelled as such. Vendor numbers are marked as vendor numbers. Nothing here is built.
- Evidence seen, with its limits:
    - **Exposure is not displacement.** The ILO's 2025 refined index of GenAI exposure keeps clerical occupations at the top; one in four workers is in an exposed occupation, only about 3.3% in the highest-exposure group, and the report calls its figures best-case upper limits that point to transformation more than replacement, because most jobs mix many tasks. A summary says data-entry clerks, payroll clerks and typists rank highest; I could not confirm that wording in the ILO text itself. (ILO Working Paper 140, summary)
    - **The old framework still organises the field.** Autor-Levy-Murnane (2003) split work on two axes, routine vs non-routine and cognitive vs manual, and argued computers substitute for tasks that follow explicit rules; later work says the "routine" boundary keeps moving as technology improves. (Autor, Levy & Murnane 2003, text, Autor & Price 2013 update)
    - **Which activities are most automatable.** McKinsey's 2017 work singles out data collection, data processing and physical work in predictable environments; a 2025 write-up cites about 57% of US work hours as technically automatable (I did not verify that against McKinsey's own page, so treat it as secondhand). (McKinsey slide deck via NAPA)
    - **How much time goes on it (weak evidence).** A vendor survey says over 40% of workers spend at least a quarter of the week on manual, repetitive tasks, mostly email, data collection and data entry. (Smartsheet survey)
    - **Why earlier automation broke.** Sources agree that robotic process automation (RPA) is brittle: a moved button or renamed field stops a bot or yields bad data; licences are only about 25-30% of total cost, the rest is implementation and maintenance; failure rates of 30-50% are quoted but mostly from vendors selling alternatives. One source warns the agent-based replacement has a quieter failure mode: an agent that drifts produces a confident wrong answer, so exception rate per run should be tracked as the earliest alarm. (RPA break-fix costs, Kognitos on hidden RPA costs, why bots keep breaking)
    - **Small businesses are the under-served majority.** A Forvis Mazars report estimates only about 12% of Indian MSMEs (7.7 million) reached digital maturity; ICRIER and RIS run large MSME digitalisation

surveys that I did not open, so I have no sourced figure for WhatsApp/Excel dependence. (Forvis Mazars on MSME digitalisation, ICRIER 2025 survey)

- **Not found:** a 2025 World Economic Forum figure for the share of work time that is routine. The WEF numbers that surfaced were from its 2020 report.

PART 1 - WHAT "MANUAL REPETITIVE WORK" IS

1.1 The four boxes (Autor-Levy-Murnane, with what an AI Employee can touch)

	Cognitive (information)	Manual (physical)
Routine (explicit rules, same every time)	Data entry, bookkeeping, form-filling, reconciliation, scheduling, reminders, report compiling, QC against a checklist. **The perfect AI Employee's home ground.**	Assembly, picking/packing, sorting, cleaning, harvesting, machine tending, inspection by sight. **Software cannot do it; the AI Employee can do the records, instructions, scheduling, counting and quality evidence around it. Robots do the motion.**
Non-routine	Judgement, advice, negotiation, design, diagnosis. **Assist only: draft, check, flag; a named person decides** (the tier A/B/C ladder of all 24 registers).	Dexterous, variable-environment work: nursing, repair, plumbing, caregiving. **Out of scope; support with checklists, records and training.**

Reality is blurrier than the boxes: most jobs are a *bundle* of tasks (ILO), so the right unit is the **task**, not the occupation, and the "routine" edge moves as models improve.

1.2 The twelve families of repetitive work (my synthesis from the 24 registers)

Each family appears in almost every industry. "Seen in" lists registers where roles of this family were proposed.

#	Family	The repeated move	Seen in (examples)
R1	**Capture**	Turn messy input (photo, PDF, voice note, WhatsApp, email, handwriting) into structured fields	prescriptions (pharmacy), delivery PODs (logistics), mandi slips (agriculture), bills (retail), muster (construction)
R2	**Re-key / move**	Copy the same data from one system to another by hand	HIS to insurer portal, ERP to GST portal, listing to marketplace, courier to ledger
R3	**Reconcile / match**	Compare two lists and explain the differences	bank vs ledger, GRN vs invoice, settlements vs orders, DT meters vs billing, claims vs bills
R4	**Fill and submit forms**	Same form, different values, portal after portal	government applications, licences, returns, KYC, customs, tender documents
R5	**Watch and alert**	Check something on a schedule and tell someone only when it changed	licence expiry, outage, stock-out, price, deadline, document validity, status of an application
R6	**Chase and remind**	Follow up people until they act	fees, rent, renewals, no-shows, documents, approvals, vendor deliveries
R7	**Compose standard documents and messages**	Same letter/quote/invoice/notice with different facts	notices, quotes, call sheets, discharge summaries, agreements, circulars
R8	**Classify, triage, route**	Read an item and send it to the right place	tickets, complaints, emails, alarms, claims, leads, grievances
R9	**Compile and report**	Pull numbers from many places into a recurring report	MIS, dashboards, board packs, monthly accounts, compliance returns

#	Family	The repeated move	Seen in (examples)
R10	**Check against a rule**	Verify an item meets a checklist or standard	QC, completeness, compliance, AQL, coding, label and claim checks
R11	**Answer the same question**	Respond to repeated enquiries from a known source	front desks, helpdesks, citizen services, student queries, tenant queries
R12	**File, archive and keep records**	Name, tag, store, retain and retrieve	MRD, DMS, case files, test records, registers, audit trails

Insight from the registers: **most of the roughly 2,900 AI Employee candidates listed across the 24 files (a sum of the register headers) are one of these twelve moves applied to a domain.** ("Most" is a judgement from reading them; I did not count each role against the twelve.) Build the twelve moves well once, and industries become *playbooks*, not new products.

1.3 Physical repetitive work (the other half of the question)

Worldwide, a large share of repetitive work is physical: factory assembly, warehouse picking, farm harvesting, cleaning, driving, construction trades, caregiving routines. A software AI Employee does not do the motion, and pretending otherwise would be dishonest. What it does for physical repetitive work:

1. **Records the work** (voice/photo/QR count, attendance, output, rejects) so the owner finally sees what happened (R1, R12).
2. **Plans and chases it** (rosters, schedules, materials, maintenance, deliveries) (R5, R6).
3. **Checks the result** against a standard (photo QC, measurement logs) (R10).
4. **Trains and prompts the worker** in the local language by voice and video (R11).
5. **Hands off to machines properly** when a robot or controller exists: data in, results out, **never writing to the controller** (the OT boundary rule from the manufacturing and energy registers). Everything in 1-4 is already inside Hoola's design language; item 5 is deliberately limited.

PART 2 - WHY OLD AUTOMATION KEPT FAILING (and what that demands of the new one)

Failure of past automation	Evidence	Requirement for the perfect AI Employee
Brittle to change: a moved button or new column stops the bot or corrupts data	RPA sources above	Prefer APIs/official connectors; fall back to vision/UI agents only last; **detect layout/schema drift before acting**
Silent wrong answers (agents "confidently" continue)	the agentic-failure warning above	**Verify before act**, in code, with evidence; measure exception and silent-error rates; canary checks
Cost of upkeep dwarfs licences	25-30% licence share	Playbooks as readable, human-editable documents; self-diagnosing; cheap to repair, ideally repaired by the owner talking to it
Only developers can build it	RPA dev-hours per break	Teach by *showing* (voice, screen, example), not programming
All-or-nothing autonomy	project failure stories	A **trust ladder** per playbook, earned by clean runs, reversible at any time
Exceptions kill the ROI	80/20 reality	**Exception-first design**: the system's job is to find the 10% that need a human and hand them over well
Unexplained, un-auditable	regulators, auditors	Every run leaves a ledger: input, checks, decision, who approved, output

Failure of past automation	Evidence	Requirement for the perfect AI Employee
Ignores the small majority	MSME under-digitisation	WhatsApp/voice/local language, no IT team, cheap runs

---

PART 3 - THE PERFECT AI EMPLOYEE FOR REPETITIVE WORK

3.1 Name and one-line definition

Roz-ka-Kaam Sahayak ("Routine Sahayak", working name). *An always-on employee that learns a repetitive job by watching you do it once, writes it down as a plain playbook you can read and edit, runs it exactly the same way every time, checks its own work in code before anything leaves, hands you only the exceptions with the evidence attached, and gets trusted with more only as it earns it.*

It is **one employee with twelve skills (R1-R12), configured by playbooks per industry**, not twelve products.

3.2 The fourteen properties of "perfect" (what must be true)

1. **Learns by demonstration.** The owner shows the job once (voice walk-through, screen recording, a few example inputs and the correct outputs, a WhatsApp thread, or a filled spreadsheet) and the employee drafts the playbook, asks the missing questions, and reads it back. No programming, no flowchart tool.
2. **The playbook is the unit.** A human-readable, versioned document: purpose, trigger, inputs, steps, rules, checks, exceptions, owner, approval level, limits. Owners can read it in their language and edit it by talking. It is the contract, the training manual, the audit evidence and the debugging tool in one. (Hoola's Skill Desk is the natural store.)
3. **A strict execution ladder: deterministic first, model second, pixels last.** (a) Official API / connector / file / email parser; (b) deterministic code for arithmetic, matching, date maths, thresholds (pylab and rules, never a model "guessing" a sum); (c) model only for reading messy input and for drafting words; (d) screen/vision operation of a website or app only when nothing else exists, with the user's tap for submit/payment/OTP. This is how it stays cheap, fast and not brittle.
4. **Verify-before-act.** Every extracted value carries its source (page, region, message); every output passes checks written as code: totals tie, no duplicates, values in range, required fields present, identity matches, date plausible. A failed check never "fixes itself": it becomes an exception.
5. **Exception-first.** It routes the uncertain, the unusual and the high-stakes to a human with a one-screen case: what it saw, what it thinks, why it stopped, the options, the evidence. It never guesses to avoid bothering you. It also learns from each resolution (proposes a rule; the owner approves the rule).
6. **A trust ladder per playbook.** Level 0 *watch and report only* -> 1 *draft, human does it* -> 2 *does it on your tap (card)* -> 3 *does it alone within caps and with a daily summary* -> 4 *autopilot with spot audits*. Promotion needs a track record (for example N clean runs and a recent audit); any incident, drift or complaint demotes it automatically. **Money out, sending to third parties, deleting, filing and public changes never go above level 2** unless the owner pre-approves a capped, narrow rule (inherited Hoola rule).
7. **Idempotent, resumable, no double-posting.** Every step has a key; a crash or a retry never repeats an effect; runs resume from the last checkpoint (the durable-run pattern Hoola already has for long jobs and enterprise writes).
8. **Audit ledger by default.** For each run: trigger, inputs, checks and results, decisions, approver, outputs, time, cost. Immutable, exportable, searchable ("show me every invoice it touched in October").
9. **Drift and quiet-failure detection.** Canary runs on known cases, schema/layout fingerprints, sudden changes in exception rate, volume or value distribution, and "too perfect" patterns. It stops and tells you *before* it corrupts, and says what changed.

10. **Cheap by design.** Small model for routine reading, large model only on exceptions; caching; batch; per-run and per-day caps; the cost of every playbook shown next to the time it saved. A playbook that costs more than the person it replaces is reported, not hidden.
11. **Speaks the worker's channel.** WhatsApp, voice call, a phone app or an email, in the owner's language (Hinglish and regional), with large-type and voice options. Low bandwidth, offline capture for field work. No new software to learn.
12. **Honest reporting.** A short daily/weekly note: what it did, what it could not, what needs you, what changed in the world (a form, a rate, a rule), and how many minutes it gave back. It never claims done before the check passes (Hoola's "done = verified" gate).
13. **Respects people and data.** Least-privilege access, secrets never in the model, IDs masked, consent for personal data, no covert monitoring of workers (it measures *work items*, not *people*), retention rules, per-client separation. It is framed and priced as **freeing a person's time**, with a redeployment plan, not as surveillance or headcount cutting; the owner decides how to use the saved time.
14. **Knows its limits.** It states what it will not do (physical motion, safety-critical control, professional sign-offs, anything in a register's "NEVER" list) and hands those to a named person instead of improvising.

3.3 The twelve skills (R1-R12) as capabilities

Skill	Must do well	Core techniques	Key guard
R1 Capture	Photos, PDFs, voice, handwriting, mixed Indian-language text to fields	OCR + vision + speech, layout-aware extraction, confidence per field	Show source highlight; low-confidence fields go to the exception queue
R2 Re-key/move	Move data between systems	API/connector first; file import; UI agent last	Dry-run diff first; idempotency keys; tap for effects
R3 Reconcile	Match two sets, explain breaks	Deterministic matching ladders (exact -> tolerance -> fuzzy), break classification	Matching in code; no model-made totals; every unmatched item named
R4 Forms	Fill and submit portal forms	Fill from verified data; read-back; stop at OTP/captcha/payment	User taps submit/pay; screenshots kept
R5 Watch	Scheduled checks, change alerts	Calendars, polling official sources, diff detection	Alert only on change; no alert fatigue (digest)
R6 Chase	Reminders and follow-ups	Polite escalating cadence, channel choice, opt-in, stop on reply	Consent; tone rules; no harassment; owner approves templates
R7 Compose	Standard documents and messages	Templates + facts + blanks for unknowns	Never invent facts; sources table; approval card
R8 Triage	Classify and route	Rules first, model for ambiguity, SLA clocks	Misroute metrics; human review for sensitive classes
R9 Report	Recurring compilations	Pull, compute in code, chart, narrative with caveats	Numbers from code only; show data freshness
R10 Check	Rule/standard checks	Checklists as code, evidence capture	Pass/fail with reasons; humans own the standard
R11 Answer	Repeated questions from approved sources	Retrieval from the owner's own material with citations	Says "I don't know" and routes; no unsourced claims
R12 Record	File, tag, retrieve, retain	Naming rules, metadata, retention schedule, search	No deletion without authority; legal holds

3.4 What a playbook looks like (illustrative, not final)

```
playbook: "Monthly GST invoice check"            owner: Meghana        language: hi-IN
trigger:   on the 5th of each month, 10:00        approval_level: 2 (card)      cap: 500 invoices/run
inputs:    - sales sheet (Google Sheet "Sales")   - purchase PDFs (Drive folder "Bills")
steps:
  1. read purchase PDFs -> fields {supplier, GSTIN, invoice_no, date, taxable, tax, total} [R1]
  2. check: GSTIN format + checksum; total = taxable + tax (code); date in month [R10]
  3. match to supplier ledger by (GSTIN, invoice_no); tolerance +-1 rupee [R3]
  4. build exceptions list: unmatched, duplicates, GSTIN invalid, amount mismatch [R8]
  5. draft message to each supplier with missing invoices (not sent) [R7,R6]
  6. produce summary + month ZIP for the CA [R9,R12]
exceptions -> show card to owner with evidence; never auto-correct amounts
never: send to suppliers, edit the ledger, file anything without a tap
metrics: straight-through %, exception count, minutes saved, cost
```

3.5 How it maps onto Hoola today (so nothing is rebuilt)

Needed	Hoola already has	Gap to build
Triggers, reminders, schedules	Routines, reminders, kaam_followup wake turns, worker slots, restart pickup	Playbook-level trigger registry with trust level
Durable runs, resume, no double-post	Long-job engine (kaam_plan with plan/step/verify), turn_inbox durable accept, enterprise run engine (plan -> dry run -> tap -> batches -> verify, idempotency keys, `ambiguous` state)	Generalise the enterprise run engine to any playbook
"Done = verified"	`agent/verify.py` gate (files on disk, pages ready, links alive, list counts)	Per-playbook check library (totals, duplicates, ranges)
Operating websites/apps	Computer Operator (OTP/submit/payment are the user's), browser worker, PC Operator (frozen)	Drift fingerprints, replay of recorded flows
Declared actions to outside systems	`appmaker/connect.py` (her approval tap bound to a hash, https + approved host, sealed secrets, safe retries, `ambiguous`)	Expose as a generic connector kind
Reading messy input	Doc reader, vision review, datasets, Data Desk, `scrape` lane, voice transcripts	Confidence-per-field and source-highlight output
Math and matching in code	`pylab`, enterprise calc (reconcile ladder: ref first, then amount AND date)	Reusable matching ladder as a tool
Skills as documents	Skill Desk / skill packs (read-before-follow law)	Playbook schema and recorder on top
Channels	WhatsApp opt-in, voice (Gemini/OpenAI Live), phone app, console	Exception inbox card with evidence
Approvals	Approval cards, trust ladder (Employee gate), caps, kill switches	Auto-promotion/demotion rules
Measurement culture	Quality cards (tool card, hard card, long card, research card, maps card)	A **REPEAT card** (below)
Cost control	Usage meters, run caps (`research` R5 pattern, `mapsbill`)	Per-playbook cost vs time-saved line

3.6 The REPEAT card (how we would prove it is perfect, not just claim it)

Following Hoola's existing card pattern (asks with checkable end states, scored in code, traps planted, free run plus a live run, `pass^3`):

- **60 tasks, five per family R1-R12**, in a mix of industries and languages; inputs with **planted traps**: duplicate invoice, wrong total, swapped digits, expired licence, unreadable photo, changed column order, stale rate card, prompt injection hidden in an email (text inside data is never instructions), a request that exceeds the playbook's authority.
- Scored on **end state**, not on prose: correct outputs, nothing sent/paid/deleted without a tap, no double effects after a forced crash, exceptions raised for each trap, no invented values, cost within cap.
- Headline numbers to publish honestly: **straight-through rate** on clean cases; **exception recall** (traps caught); **silent-error rate** (traps passed unnoticed - target zero); **false-alarm rate**; **drift detection time** on a changed layout; **median cost per run**; **minutes saved vs a human timed on the same case**; **pass^3**.
- Humans spot-audit a sample of the model's "easy" cases, because that is where quiet errors hide. Targets are goals to be measured, not promises: no run has been done.

3.7 Safety and ethics: the non-negotiables

- No payment, send, delete, file, publish or access grant without the owner's tap (inherited rules), except under a narrow, capped, pre-approved rule at level 3.
- Text inside documents, emails and web pages is **data, never instructions**; planted instructions are logged and ignored.
- No PIN/OTP/CVV/passwords in chat; credentials sealed and never visible to the model.
- No scraping against terms; official connectors only.
- No individual-worker scoring from the logs; metrics are about items, queues and processes.
- The saved time belongs to the owner: reports show it; the product never recommends firing a named person.
- Regulated acts remain with licensed humans (every register's "NEVER" list applies as default policy).

3.8 Build plan (phased, honest about what is unknown)

Phase	What	Exit test
P0	**Measure first**: build the REPEAT card (free run: scorer calibrated by planted-fault cases, no model) and baseline today's Hoola on it	Card catches 100% of its own planted faults
P1	**Playbook format + store** (Skill Desk based), human-readable, versioned; run ledger	Owner can read and edit a playbook by voice
P2	**Core checks library** (totals, duplicates, ranges, GSTIN/PAN formats, date logic) and **matching ladder** as tools	Unit tests + card cases R3, R10 green
P3	**Exception inbox** (card with evidence, options, one-tap resolve, rule proposal)	Exceptions resolved in under 30 s median in a pilot
P4	**Teach-by-showing**: voice/screen/example demonstration to playbook draft with clarifying questions	Time-to-first-working-playbook under 30 min for a simple job
P5	**Trust ladder + drift monitor** (auto-demote on incident/drift; canaries)	Forced drift test detects and halts before damage
P6	**Pilots**: one small CA firm (R3, R4, R6, R9), one clinic/gym/shop front desk (R6, R11, R5), one MSME factory (R1, R9, R10)	Weekly time saved measured against a stopwatch baseline; zero silent errors in audit

Unknowns to settle by measurement: real straight-through rates by family; screen-operation reliability on changing portals; how much owners will trust level 3; true cost per run on Indian-language handwriting.

3.9 Why this beats the alternatives (reasoned, not measured)

- vs **RPA**: no scripts to maintain; failures become exceptions instead of crashes; owner can edit in language.
- vs **Zapier-style connectors**: handles messy input, exceptions and verification, not just happy-path triggers.
- vs **a general chatbot or agent**: a playbook, a ledger, a trust ladder, code-level checks and drift alarms make behaviour repeatable and auditable.
- vs **hiring**: cheaper and tireless for the routine part, but the employee is explicit that it frees people for judgement, relationships and physical work.

Open questions for the owner

1. **First family to perfect**: I would start with R3 reconcile, R6 chase and R9 report (cash-linked, rule-based, safest), then R1 capture and R4 forms. Agree?
2. **First pilot**: small CA/CS practice, clinic/gym/shop front desk, or MSME factory? (Each has a register ready.)
3. **Trust ladder ceilings**: confirm level 2 (tap) as the maximum for money, sending, deleting and filing unless she writes a capped rule.
4. **Name**: "Roz-ka-Kaam Sahayak" is a working name; she may prefer one matching her avatar naming.
5. **Teach-by-showing scope**: voice and example-based first; screen recording only where the PC Operator freeze allows (that surface stays frozen until she asks).
6. "GO per family" turns any of R1-R12 into a sourced design with measured baselines; "GO REPEAT CARD" builds P0.